

1. Describa en detalle el proceso llevado a cabo para resolver el problema planteado, indicando las decisiones tomadas en cada parte de la práctica.

El problema exigía cambiar el modelo mental imperativo por dos enfoques declarativos distintos para procesar un código de 8 dígitos. En la Parte A (Haskell), la decisión principal fue estructurar la solución como una tubería de transformación de datos mediante la composición de funciones puras, sin recurrir a estados mutables. Reemplazamos los bucles iterativos por listas por comprensión para la suma alícuota y usamos guardas para clasificar las categorías matemáticas. En la Parte B (Prolog), el enfoque cambió a modelar relaciones entre el código y sus características. La decisión arquitectónica clave fue implementar controles de instanciación (var/1 y nonvar/1) para aislar el flujo aritmético, permitiendo que el programa no solo verifique códigos, sino que también actúe como un generador de códigos válidos.

Sobre la solución desarrollada en Haskell (paradigma funcional):

2. Describa una forma en la que el proceso de currificación se manifiesta en la solución propuesta.

En Haskell, las funciones están currificadas por defecto, lo que significa que evalúan un parámetro a la vez devolviendo nuevas funciones. Esto se manifiesta claramente en nuestra función `analizarCodigo` al utilizar funciones del preludio como `take`. Por ejemplo, en la expresión `take 3 codigo`, la función `take` (cuya firma es `Int -> [a] -> [a]`) recibe primero el entero 3. Al evaluarse, no retorna un valor final, sino que devuelve una función parcialmente aplicada que espera recibir una lista de caracteres (el `string`). Finalmente, esa nueva función se aplica a `codigo` para devolver los primeros 3 caracteres.

3. Indique, para cada función desarrollada, si la identifica como polimórfica, de orden superior, o si no la considera clasificada bajo ninguno de los dos criterios; justifique cada determinación.

`esCodigoValido (String -> Bool)`: Es una función de orden superior porque en su cuerpo utiliza la función `all`, la cual recibe por parámetro otra función (`isDigit`) para aplicarla sobre la cadena. No es polimórfica exteriormente ya que opera estrictamente sobre `String`.

sumaAlicuota (Int -> Int): No se clasifica como polimórfica ni de orden superior en su firma. Opera estrictamente sobre enteros y retorna un entero, aunque internamente delega la reducción a la función de orden superior sum.

extraerPeriodo, formatearConsecutivo, determinarParidad y clasificarCategoria: Son funciones de primer orden y monomórficas. Se diseñaron intencionalmente con restricciones de tipo rígidas (Int y String) para cumplir con las validaciones exactas del dominio del problema, por lo que no operan sobre tipos genéricos.

Sobre la solución desarrollada en Prolog (paradigma lógico):

4. Elija uno de los predicados desarrollados y escriba su lectura declarativa y su lectura procedimental. Explique en qué situaciones ambas lecturas no coinciden y qué consecuencias tiene esto para el programador.

Analizando el predicado: evaluar_categoria(N, Suma, 'Administrative') :- N < Suma, !.

Lectura declarativa: "La categoría de un número N es 'Administrative' si y solo si N es estrictamente menor que su suma alícuota".

Lectura procedimental: "Para satisfacer el objetivo de clasificar N como 'Administrative', primero evalúe aritméticamente si N es menor que Suma. Si tiene éxito, ejecute el corte (!) para comprometerse con esta regla y descartar las demás".

Divergencia: Las lecturas dejan de coincidir debido al operador de corte (!).

Declarativamente, el corte no tiene ningún significado lógico. Procedimentalmente, altera drásticamente el árbol de resolución. Si el programador ordena mal las cláusulas, Prolog podría fallar en encontrar una verdad lógica válida porque el corte pudo esa rama de búsqueda prematuramente.

5. Explique cómo participan la unificación y el backtracking en el cálculo de la suma alícuota. Indique si usó el operador de corte (!), en qué puntos y qué efecto tendría su eliminación sobre el conjunto de respuestas.

El backtracking es el motor de búsqueda en el predicado es_divisor. Al usar between, Prolog instancia una variable con un número, evalúa si el módulo es 0 y, de no serlo, retrocede iterativamente probando las siguientes alternativas. La unificación ocurre cuando el predicado de agregación findall/3 recolecta todas esas alternativas exitosas y las unifica en una única lista para ser sumada.

El operador de corte (!) se usó explícitamente en evaluar_categoria y paridad para garantizar que las reglas sean mutuamente excluyentes. Si se eliminara, el programa

seguiría arrojando respuestas correctas (porque las condiciones matemáticas $<$, $=$, $>$ ya son excluyentes entre sí), pero perdería mucha eficiencia. Prolog dejaría "puntos de elección" abiertos en la memoria e intentaría evaluar inútilmente las demás reglas incluso después de haber clasificado el número con éxito.

6. Determine cuáles de sus predicados son reversibles, es decir, cuáles admiten más de un modo de instanciación, y cuáles no. Justifique en cada caso qué impide la reversibilidad e incluya la consulta ejecutada en modo "generar" junto con las respuestas obtenidas.

El predicado `clasificar_nicomaco` es reversible porque implementamos un diseño explícito con `var/1` y `nonvar/1`. Puede recibir un número para verificar su categoría, o recibir una categoría para generar números que cumplan dicha condición matemática. Los predicados que no son reversibles son aquellos que dependen puramente de evaluación aritmética inmediata (como `es_divisor` con `mod =:= 0` o las evaluaciones en `codigo_caracteristicas` como `P is Codigo div 100000`). En Prolog, el operador `is` y los comparadores aritméticos exigen que las variables del lado derecho estén completamente instanciadas antes de ejecutarse, bloqueando la reversibilidad si se usan en modo generar sin una acotación previa (como `between`).

Consulta en modo generar (Códigos del periodo 2029-2 en Ingeniería):

?- `codigo_caracteristicas(Codigo, '2029-2', 'Engineering', _, _)`.

Respuestas obtenidas:

`Codigo = 29206001 ;`

`Codigo = 29206002 ;`

`Codigo = 29206003 ...`

(El sistema genera iterativamente los consecutivos válidos asociados a las categorías matemáticas perfectas como el 06 o el 28).

7. Compare ambas soluciones e indique qué partes del problema se expresaron de forma más natural en el paradigma funcional y cuáles en el paradigma lógico.

Identifique al menos un aspecto de la solución en el que el grado de declaratividad sea mayor en un paradigma que en el otro, y justifique su respuesta.

La manipulación de los datos del código (fragmentar el periodo, la categoría y el consecutivo) se expresó de forma mucho más natural en el paradigma funcional (Haskell). Las operaciones como take y drop permiten establecer un flujo de transformación de cadenas muy directo y legible, y el pattern matching facilita la asignación de periodos.

Por otro lado, el grado de declaratividad fue notablemente mayor en el paradigma lógico (Prolog) al momento de buscar los divisores de un número. En Haskell, se requirió escribir la mecánica explícita para armar la colección: $[i \mid i \leftarrow [1..(n-1)], n \bmod i == 0]$. En Prolog, gracias a la agregación con findall/3, bastó con declarar el "hecho" de qué constituye ser un divisor (es_divisor) y delegar al motor de inferencia la recolección automática de todas las verdades posibles mediante backtracking. Nos desentendimos del "cómo" agruparlos, logrando una expresión puramente declarativa del conocimiento matemático.